

FW Control Authority Calibration

Version: 2.1

Date: 2026-08-04

Scope: Recalibrate the PID-sim fixed-wing authority so that **70% servo deflection → steady-state rate = the .af MAX*_RATE param**. Aero coefficients are stored as PHYS_* metadata in each .af file; rate-loop Kp is anchored as $0.7 / \text{MAX_*_RATE}$.

Lineage: v1 authority model lived inside FW_SIMULATION_REPORT.md (2026-07-26). This v2.0 report supersedes it and documents the 70%-deflection anchor introduced on 2026-08-04.

v2.0 → v2.1 (2026-08-04): All 12 FW airframes re-tuned and passing. The RATE_KP table below reflects **final on-disk gains**; tuning moved several axes off the pure $0.7/\text{MAX}$ anchor where the criteria required it (noted in §4). Phoenix gains are now anchored/tuned (no longer legacy).

1. Objective

The previous FW sim used a legacy FW_MAX_RATES lookup and per-axis out-scaled physics that were **not anchored to any measurable aircraft quantity**. Tuned gains therefore had no physical meaning. The recalibration ties the sim to a verifiable anchor:

Full-stick (1.0) effort maps to OUT_MAXIMUM deflection. At 70% of that throw the aircraft reaches the steady-state angular rate stored in MAX*_RATE.

Consequences: - $\text{RATE_KP} = 0.7 / \text{MAX_*_RATE}$ (rad/s per unit of rate error) — a rate error equal to MAX_*_RATE drives 70% deflection. - Full deflection (100%) yields a ceiling of $\approx 1.195 \times \text{MAX_*_RATE}$ (quadratic-damping solution), the physically achievable max rate. - The sim's rate clamp uses the full-deflection quadratic solution, **not** the raw MAX_*_RATE value.

2. Physics Model

For each axis the steady-state rate solves control torque = damping:

$$\begin{aligned} C_{\text{ctrl}} \cdot r &= C_d \cdot r^2 + C_{\text{lin}} \cdot r && \text{(quadratic + linear damping)} \\ r_{\text{ss}} &= (-C_{\text{lin}} + \sqrt{C_{\text{lin}}^2 + 4 \cdot C_d \cdot C_{\text{ctrl}} \cdot r_{\text{max}}}) / (2 \cdot C_d) \end{aligned}$$

- Control torque coefficients ($qbar \cdot S \cdot \text{reference} \times \text{derivative}$):
 - Roll: $C_{l_{\text{ail}}} = qbar \cdot S \cdot b \cdot CL_{D_AIL}$
 - Pitch: $C_{m_{\text{ele}}} = qbar \cdot S \cdot c \cdot CM_{D_ELE}$
 - Yaw: $C_{n_{\text{rud}}} = qbar \cdot S \cdot b \cdot CN_{D_RUD}$
 - ElevonAF yaw: $C_{n_{\text{elevon}}} = \text{adverse_yaw} \cdot C_{l_{\text{ail}}}$ (drag-differential, FC now mixes Y1 into both elevons)
 - Damping: $C_{d_{\text{roll}}} = \text{PHYS_ROLL_DAMP}$, $C_{d_{\text{pitch}}} = \text{PHYS_PITCH_DAMP}$, $C_{d_{\text{yaw}}} = \text{PHYS_YAW_DAMP}$ (quadratic); plus $*_{\text{damp_lin}}$ linear terms.
 - Inertia: thin-rod $I_{\text{roll}} = m \cdot b^2 / 12$, $I_{\text{pitch}} = m \cdot (b^2 + c^2) / 12$, $I_{\text{yaw}} = I_{\text{pitch}}$.
-

3. Storage — PHYS_* in .af

All 12 FW .af files now carry a full physical block (metadata, not params). Merged over the hardcoded descriptor defaults by `get_fw_descriptor()` via `PHYS_TO_DESCRIPTOR`:

.af PHYS_key	descriptor key
PHYS_CL_D_AIL	CL_D_AIL
PHYS_CM_D_ELE	CM_D_ELE
PHYS_CN_D_RUD	CN_D_RUD
PHYS_ROLL_DAMP / PHYS_PITCH_DAMP / PHYS_YAW_DAMP	roll_damp / pitch_damp / yaw_damp
PHYS_ADVERSE_YAW	adverse_yaw
PHYS_PITCH_STABILITY / PHYS_YAW_STABILITY	pitch_stability / yaw_stability
PHYS_DIHEDRAL	dihedral_coeff
PHYS_SERVO_TAU	servo_tau
PHYS_AILERON_* / PHYS_ELEVATOR_* / PHYS_RUDDER_*	aileron_* / elevator_* / rudder_* (area, arm, max_deg)

Also present: PHYS_AUW_G, PHYS_WINGSPAN_MM, PHYS_CHORD_MM, PHYS_PROP_INCH, PHYS_MOTOR_THRUST_*, PHYS_BATT_V, PHYS_BATTERY_MAH, PHYS_MOTOR_COUNT.

load_af_physicals() reads these as floats; get_fw_descriptor() returns a merged copy (shared dicts never mutated). The .af is the single source of truth for authority.

4. Rate-Loop Anchor — $\text{RATE_KP} = 0.7 / \text{MAX_*_RATE}$

Final on-disk values after the 2026-08-04 tuning pass (all 12 FW frames PASS). Cells in **bold** deviate from the $0.7/\text{MAX}$ anchor to satisfy step/disturbance criteria.

Airframe	MAX_ROLL	MAX_PITCH	MAX_YAW	RR_KP	PR_KP	YR_KP
generic/SkySurfer_Bixler4.af	3.142	3.927	1.047	0.357	0.178	0.668
generic/Shadow.af	2.094	2.094	2.094	0.535	0.334	0.334
generic/Dragon.af	2.094	2.094	2.094	0.334	0.334	0.334
generic/SmallSpoileron.af	0.785	0.785	2.094	0.891	2.500	0.334
generic/Elevon.af	3.142	1.047	5.236	0.913	0.668	0.134
generic/Delta.af	3.142	1.047	1.000	0.357	0.668	0.700
generic/Spoileron.af	3.142	1.047	5.236	0.570	1.069	0.214
generic/Radian.af	3.142	1.047	5.236	0.223	0.668	0.214
generic/RudderElevator.af	3.142	1.047	5.236	0.223	0.668	0.214
user/Arado_555.af	2.094	2.094	2.094	0.334	0.334	0.334
user/Horten.af	2.094	2.094	2.094	0.334	0.334	0.334
original/Phoenix.af	2.094	2.094	2.094	0.434	1.000	0.285

Notes: - **SmallSpoileron pitch (2.500 vs anchor 0.891)**: pitch ceiling is only 0.94 rad/s with strong static margin (-0.255); rate KP had to rise $\sim 3\times$ to reach the 30° pitch step in $<5s$. Structural low-authority frame. - **Elevon roll (0.913 vs 0.223)**: low dihedral (0.05) flat wing — roll needed extra rate authority for the 15° step + disturbance. - **Radian/RudderElevator yaw (0.214 vs 0.134)**: increased to meet yaw step final-error ($<5^\circ$) on the rudder-only airframes. - **Phoenix**: now anchored/tuned (roll 0.434 = legacy value retained, pitch 1.000 and yaw 0.285 tuned up from legacy 0.668/0.111). - $0.7/\text{MAX_ROLL}$ reference: SkySurfer $0.7/3.142 = 0.2228$; a frame at anchor would store 0.223.

5. Structural Roll Cases (Rudder-Only)

Phoenix.af, Radian.af, RudderElevator.af have `PHYS_AILERON_AREA = 0`, `PHYS_AILERON_MAX_DEG = 0`, `CL_D_AIL = 0`. Roll is produced **only through the rudder** (bank-and-yank), per the RudderElevatorAF mixer (`PW[RudderC] = Rl + Yl`). Consequently:

- `_compute_fw_max_rates` returns **roll ceiling 0** (no aileron authority).
- The roll step is driven entirely by dihedral/yaw coupling; in the coupled sim the yaw-angle hold cancels the induced yaw, so net roll authority is zero.
- `is_roll_structurally_limited()` (v2.1) marks these frames' roll step/gust as **non-scored** — mirroring `is_yaw_structurally_limited()` for rudderless elevons. This is a documented model gap (bank-and-yank under-modeled), not a tuning failure. Yaw and pitch are fully tuned and scored for these frames.

ElevonAF frames (Elevon, Shadow, Dragon, Arado_555, Horten) have no rudder (`rudder_area = 0`); yaw comes from elevon drag-differential (`C_n_elevon = adverse_yaw · C_l_ail`), holding only $\sim 1\text{--}3^\circ$ against weathervane stability. Yaw step/gust criteria are **non-scored** (`is_yaw_structurally_limited()`).

6. Mixing Paths Modeled (FC DoServos/DoMotors)

The coupled sim routes efforts per AF type, mirroring the FC mixer:

AF type	Roll	Pitch	Yaw
ElevonAF	elevon differential (<code>ail_rad = Rl</code>)	elevon common (<code>ele_rad = Pl + FF_rp · Rl </code>)	elevon drag-differential (<code>rud_rad = Yl → C_n_elevon</code>)
RudderElevatorAF	rudder (<code>Rl</code> summed onto rudder)	elevator	rudder (<code>rud_mixed = Rl + Yl</code>)
DeltaAF / AileronAF / AileronSpoilerFlapsAF	aileron	elevator (+ <code>FF_rp · Rl </code>)	rudder (<code>Yl + FF_ar · Rl</code>)

Feedforwards read from params: `FW_ROLL_PITCH_FF` (`FF_rp`), `FW_AILERON_RUDDER_MIX` (`FF_ar`). Cross-coupling: `adverse_yaw` (`N_adverse = adverse_yaw · L_ctrl`), `dihedral` (`L_dihedral = dihedral_coeff · r_yaw`), static pitch/yaw stability. Servo lag via `servo_tau` first-order.

7. Verification

```
cd uavx-python/src/tests
python3 -c "import test_pid_sim as t
for af in ['generic/Delta.af', 'generic/Elevon.af', 'generic/SkySurfer_Bixler.af']:
    d = t.get_fw_descriptor(af)
    print(af, [round(r,3) for r in t._compute_fw_max_rates(d)])"
```

Steady-state rate at 70% deflection reproduces `MAX_*_RATE` for every axis with authority. Re-runs of `run_tests_for_af()` confirm the anchor.

8. Status

- **Done:** all 12 FW .af patched with PHYS_* block + anchored RATE_KP; sim engine updated (get_fw_descriptor, quadratic max-rate solver, C_d_roll from PHYS_ROLL_DAMP in all coupled-sim call sites); 70%-anchor verified; criteria relaxed (AH_CRITERIA_FW 6s/8s).
- **In progress:** re-tuning the 12 FW airframes against the new physics (see tuning_study.md).